B3 — Transações e controle de concorrência

ToyStorySystem · Laboratório de Banco de Dados · MySQL / InnoDB

1. Operação transacional escolhida

A operação documentada é o **cadastro de cliente**, implementada no método **inserir** da classe CRUDCliente (br/com/entidade/CRUDCliente.java) e acionada pela tela InterfaceCliente. Ela grava em **duas tabelas: pessoa e cliente**.

Na tabela cliente, id_pessoa é ao mesmo tempo chave primária e chave estrangeira de pessoa. Por isso o INSERT em pessoa precisa vir primeiro: ele gera o id (AUTO_INCREMENT), que é usado no INSERT em cliente. Se o segundo comando falhar (por exemplo, CPF duplicado, restrição uq_cliente_cpf), o primeiro não pode permanecer gravado, pois restaria uma pessoa sem cliente. As duas gravações devem acontecer juntas ou nenhuma delas (atomicidade).

2. Como a transação é controlada no código

Etapa	No código
Início	con.setAutoCommit(false), que desliga o commit automático e inicia a transação.
Comando 1	INSERT INTO pessoa(nome, status) VALUES (?, ?), com PreparedStatement; o id gerado é recuperado com getGeneratedKeys().
Comando 2	INSERT INTO cliente(id_pessoa, cpf, data_nascimento, pontos_fidelidade) VALUES (?, ?, ?, ?), usando o id do passo anterior.
COMMIT	con.commit(), executado somente se os dois comandos funcionaram.
ROLLBACK	Em qualquer exceção, o catch chama desfazer(), que executa con.rollback(); a exceção é relançada para a interface.
Encerramento	O finally chama finalizar(): devolve setAutoCommit(true) e fecha a conexão.

Trecho resumido de CRUDCliente.java:

```
public int inserir(Cliente c) throws Exception {
    try {
        abrirBanco();
        con.setAutoCommit(false);   // início da transação

        String sqlPessoa = "INSERT INTO pessoa(nome, status) VALUES (?, ?)";
        // PreparedStatement com RETURN_GENERATED_KEYS -> id = id gerado

        String sqlCliente = "INSERT INTO cliente(id_pessoa, cpf,
                data_nascimento, pontos_fidelidade) VALUES (?, ?, ?, ?)";
        // PreparedStatement com id, cpf, data de nascimento e pontos

        con.commit();               // COMMIT
        c.setId(id);
        return id;
    } catch (Exception e) {
```

```
            desfazer();                 // ROLLBACK (con.rollback())
            throw e;
        } finally {
            finalizar();                // setAutoCommit(true) e fecha a conexão
        }
    }
```

Todos os valores entram por parâmetros (?) em PreparedStatement, sem concatenação de String no SQL.

3. Teste 1 — cenário de sucesso (COMMIT)

Foi cadastrado o cliente denji, CPF 69696969696. As duas inserções foram confirmadas: a contagem de clientes passou de 100.051 para 100.052 e o registro aparece com id_pessoa 200002.

	clientes
▶	100051

Figura 1 — SELECT COUNT(*) em cliente antes do cadastro: 100.051.

	clientes
▶	100052

Figura 2 — SELECT COUNT(*) em cliente depois do cadastro: 100.052.

	id_pessoa	nome	cpf
▶	200002	denji	69696969696

Figura 3 — Registro criado: id_pessoa 200002, nome denji, CPF 69696969696.

4. Teste 2 — cenário de falha (ROLLBACK)

Foi tentado o cadastro de um novo cliente reutilizando o CPF 69696969696. O INSERT em pessoa é executado, mas o INSERT em cliente viola a restrição uq_cliente_cpf (erro 1062 do MySQL). O catch chama desfazer(), e o ROLLBACK desfaz também o INSERT em pessoa. Nada fica gravado.

Na interface (B1), o código de erro 1062 é traduzido para a mensagem “Já existe um cliente com esse CPF.”, sem exibir a exceção bruta do driver.

```
1 •  SELECT COUNT(*) AS pessoas FROM pessoa;
2 •  SELECT COUNT(*) AS clientes FROM cliente;
```

Result Grid | Filter Rows: | Export:

clientes
100052

Figura 4 — Contagem de clientes após a tentativa com CPF duplicado: permanece 100.052.

5. Teste 3 — concorrência (bloqueio de linha)

Para demonstrar o controle de concorrência foram usadas duas sessões no MySQL Workbench atuando sobre o mesmo cliente. A sessão A abriu uma transação e alterou a linha sem confirmar. A sessão B tentou alterar a mesma linha (UPDATE em cliente, campo pontos_fidelidade) e ficou esperando.

trx_id	trx_state	sessao	esperando_s	consulta
56271	LOCK WAIT	25	19	UPDATE cliente SET pontos_fidelidade = pontos...
56268	RUNNING	17	NULL	NULL
284294398091544	RUNNING	13	NULL	EXPLAIN analyze SELECT pr.nome, SUM(i.quant...
284294398090768	RUNNING	12	NULL	EXPLAIN analyze SELECT pr.nome, SUM(i.quant...
284294398089992	RUNNING	10	NULL	EXPLAIN analyze SELECT pr.nome, SUM(i.quant...

Figura 5 — Transações ativas no InnoDB: a sessão 25 está em LOCK WAIT há 19 s executando UPDATE em cliente, enquanto a sessão 17 mantém uma transação aberta. As linhas EXPLAIN são outras sessões em execução, que não aguardam bloqueio.

Isso acontece porque, no InnoDB, um UPDATE obtém bloqueio exclusivo sobre a linha, mantido até o COMMIT ou ROLLBACK da transação (aula de Controle de Concorrência). A sessão B só prossegue quando a A decide; se a espera passar do tempo limite (innodb_lock_wait_timeout), o MySQL retorna o erro 1205. Assim, duas alterações simultâneas na mesma linha são serializadas e nenhuma atualização é perdida.

O nível de isolamento padrão do InnoDB é REPEATABLE READ, que mantém uma visão consistente das leituras dentro da transação.

6. Conclusão e boas práticas aplicadas

- Atomicidade: o cadastro de cliente grava em pessoa e cliente em uma única transação, com COMMIT em caso de sucesso e ROLLBACK em qualquer falha.
- Isolamento: o bloqueio de linha do InnoDB impede que duas sessões alterem o mesmo registro ao mesmo tempo, serializando as operações.

- Transação curta: apenas os dois comandos necessários ficam dentro da transação, reduzindo o tempo de posse dos bloqueios.
- Tratamento de falhas: erros de restrição (1062), deadlock (1213) e tempo de espera (1205) devem ser tratados na aplicação, com mensagem compreensível ao usuário.
- Encerramento seguro: o bloco finally devolve o commit automático e fecha a conexão, mesmo após um ROLLBACK.